

A Project Report

On

AI Career Compass

*Submitted in partial fulfillment of the
requirement for the award of the degree of*

BACHELOR OF COMPUTER APPLICATION



DEGREE

Session 2025-26

in

[Computer Application with Artificial Intelligence and Machine Learning]

By

Group:-BCA3292

Under the Guidance of

Mrs. Ranjeeta Mittal

**SCHOOL OF COMPUTER APPLICATIONS AND
TECHNOLOGY GALGOTIAS UNIVERSITY, GREATER**

**NOIDA
INDIA**

**Jan,
2026**

DECLARATION

I/We hereby declare that the project report entitled “AI Career Compass” submitted to **School of Computer Applications & Technology, Galgotias University** in partial fulfillment of the requirement for the award for the degree of **Bachelor of Computer Application with (AIML)** , is an authentic and original work carried out by us.

The matter embodied in this project is a genuine work done by us and has not been submitted whether to this institute or to any other University / Institute for the fulfillment of the requirements of any course of study.

Wherever We have used materials (data, mathematical analysis and text) from other sources or have quoted written materials, We have given due credit to them by giving their details in the references section.

The matter presented in the thesis/project/dissertation has not been submitted by me/us for the award of any other degree of this or any other places.

Student Names:-

Aditya Raj Pathak(23SCSE1430122)

Avinash Singh(23SCSE1430196)

Aditya Parmar(23SCSE1430162)

(Signature of approving Guide name)

Mrs. Ranjeeta Mittal

CERTIFICATE

This is to certify that Project Report entitled “AI Career Compass.” which is submitted by Aditya Raj Pathak in partial fulfillment of the requirement for the award of degree BCA. in Department of Computer Applications, of School of Computer Applications and Technology, Galgotias University, Greater Noida, India is a record of the candidate own work carried out by him/them under my supervision. The matter embodied in this thesis is original and has not been submitted for the award of any other degree

Signature of Examiner(s)

Signature of Supervisor(s)

Date: April, 2026

Place: Greater Noida

Table of Contents

Serial No.	Particulars	Page No.
A	Declaration	1
B	Certificate	2
C	Table of Contents	3
1.	Introduction (Preliminary Project Plan)	4
	Problem statement	4
	Objective	4
	Proposed Methodology	5
	Expected Outcome	6
2.	Technical Requirements & Design	7
	Libraries, Modules & dependencies	7
	Software requirements	8
	Hardware requirements	8
	System Architecture & Workflow	9
3.	Work Progress	10-13
4.	Final product: Output screens	14-15
5.	Conclusion & Future Discussions	16-17
6.	Bibliography & References	18
Appendices [if applicable]		

1.Introduction (Preliminary Project Plan)

1.1 Problem Statement

Career guidance after higher secondary education and graduation remains a significant challenge for students in India. Despite having skills and interests, many students lack access to personalised counseling that considers their academic background, technical skills, financial constraints, and preferred learning mode simultaneously. Existing systems rely on generalised advice or basic aptitude tests that fail to account for the complexity of modern career pathways.

Furthermore, the sheer volume of career options — spanning technology, healthcare, business, engineering, arts, and more — makes it impossible for students to evaluate all options manually. There is a clear need for an intelligent, data-driven system that can analyse a student's complete profile and provide ranked, personalised career recommendations backed by real NIRF-ranked college suggestions and job market data.

1.2 Objective

The main objectives of this project are:

- To design and develop an intelligent career guidance system using machine learning that accepts a student's skills, interests, academic marks, budget, and study mode preferences as input.
- To collect, clean, and merge multiple real-world datasets including NIRF Rankings 2024 (470 institutions), career survey data (1,195 responses), and fresher job listings (845 positions).
- To apply text vectorisation (TF-IDF) and train three machine learning classification algorithms — Random Forest, Logistic Regression, and Linear SVC — on the processed data.
- To evaluate model performance using Top-1, Top-3, and Top-5 accuracy metrics and select the best-performing model for deployment.
- To build a FastAPI REST backend with 8 endpoints that serves career predictions, matched NIRF colleges, eligibility tags, and relevant job listings in under 50 milliseconds per request.

1.3 Proposed Methodology

The development followed a three-phase pipeline approach:

- **Phase 1 — Data Collection and Preprocessing:** Four raw CSV datasets were downloaded and cleaned using pandas. Career labels were standardised from 381 messy variants to 22 clean categories. NIRF college fields were mapped to career domains. Fresher job salaries were parsed from inconsistent formats into annual INR figures. All cleaned outputs were saved as new CSV files for use in later phases.
- **Phase 2 — Machine Learning Model Training:** The merged career dataset (741 rows) was used to train three classification models inside scikit-learn Pipelines with TF-IDF vectorisation. Models were evaluated on a held-out test set (20%). The best model (Logistic Regression, Top-1: 45.3%, Top-3: 65.1%) was saved as `career_model.pkl` using Python's pickle library.
- **Phase 3 — Backend API Development:** A FastAPI application (`main.py`) loads the trained model and cleaned datasets at startup. It exposes 8 REST endpoints that accept student profile inputs, run ML inference, filter colleges and jobs based on preferences, and return structured JSON responses. Pydantic ensures all inputs are validated before the ML model is called.

1.4 Expected Outcome

The expected outcomes of this project are:

- A functional, end-to-end career guidance system with a trained ML model serving real-time career predictions via a REST API.
- Personalised Top-3 career recommendations with confidence scores for each student profile based on their skills and interests.
- Matched NIRF-ranked college recommendations filtered by career domain, city preference, budget, study mode, and academic eligibility.
- Relevant fresher job listings sorted by career relevance and city preference, with standardised salary information.
- An interactive API documentation interface (Swagger UI at /docs) enabling examiners and reviewers to test all endpoints live without writing any code.
- A scalable system architecture that can be enhanced with additional data sources, BERT-based embeddings, or a full frontend application in future iterations.

2. Technical Requirements & Design

2.1 Libraries, Modules & Other Dependencies

The project utilises a carefully selected set of Python libraries across data preprocessing, machine learning, and backend API development. Every library was chosen for a specific technical reason

Library	Type	Purpose in This Project	Install Command
FastAPI	Web Framework	Main server framework. Receives HTTP requests, validates inputs, routes to endpoints, serialises JSON responses. Auto-generates Swagger UI at /docs.	pip install fastapi
Uvicorn	ASGI Server	HTTP server that runs the FastAPI app. Listens on port 8000 and handles async connections.	pip install uvicorn
Pydantic	Data Validation	Defines StudentProfile data model with 11 typed fields. Automatically rejects malformed input before ML model is called.	Included with FastAPI
Pandas	Data Manipulation	Loads cleaned CSVs into DataFrames at startup. Filters 470 colleges and 845 jobs per request.	pip install pandas
NumPy	Numerical Computing	Sorts 22 career probability scores using np.argsort() to extract Top-N careers in confidence order.	pip install numpy
Scikit-learn	ML Library	Provides TfidfVectorizer, LogisticRegression, Pipeline, LabelEncoder. Core ML stack.	pip install scikit-learn
Pickle	Serialisation	Loads career_model.pkl (~2 MB) and label_encoder.pkl (~5 KB) at server startup. Python built-in.	Built-in (no install)
re	Regex	Cleans student input text before vectorisation. Removes special characters and collapses spaces.	Built-in (no install)
os	OS Interface	Verifies .pkl and .csv files exist at startup. Raises clear errors if files are missing.	Built-in (no install)
Matplotlib	Charting	Generates bar chart comparing Top-1/3/5 accuracy across 3 ML models (saved as model_results.png).	pip install matplotlib

requirements.txt (exact versions used)

```
fastapi==0.111.0 | uvicorn[standard]==0.29.0 | pandas==2.2.2 | numpy==1.26.4 | scikit-learn==1.4.2
| pydantic==2.7.1 | python-multipart==0.0.9
```

2.2 Software Requirements

The following software components are required to develop and run the system:

- Operating System: Windows 10/11, Linux (Ubuntu 20.04+), or macOS
- Python Version: Python 3.8 or higher (Python 3.11 recommended for best compatibility)
- Web Browser: Google Chrome / Microsoft Edge (for Swagger UI testing at localhost:8000/docs)
- Code Editor: Visual Studio Code with Python extension, or PyCharm
- Package Manager: pip (included with Python 3.8+)
- ML Environment: Local Python environment or Google Colab for model training
- API Testing: Swagger UI (auto-generated), Postman, or curl command line

2.3 Hardware Requirements

The system operates efficiently on standard computing hardware. No GPU is required for inference:

- Processor: Intel i3 / AMD Ryzen 3 or higher (i5/Ryzen 5 recommended for faster model training)
- RAM: Minimum 4 GB (8 GB recommended — pandas loads 470 colleges + 845 jobs into memory)
- Storage: Minimum 2 GB free disk space for datasets, model files, and Python packages
- Internet Connection: Required for initial pip installs and Kaggle dataset downloads only
- Device: Laptop or Desktop. No cloud infrastructure required for local deployment

2.4 System Architecture & Workflow

The system follows a clean three-phase pipeline. Each phase produces output files consumed by the next:

Phase	File	Purpose
Phase 1	data_preprocessing.py	Cleans and merges 4 raw CSV datasets into 4 clean output files
Phase 2	ml_model.py	Trains 3 ML algorithms, picks the best (Logistic Regression), saves career_model.pkl and label_encoder.pkl
Phase 3	main.py (Backend)	Loads trained model, exposes 8 REST API endpoints, handles predictions and data lookups

The end-to-end workflow for a student using the system is:

- Step 1 — Student submits skills, interests, marks, budget, and city preference to the /predict API endpoint via POST request.
- Step 2 — FastAPI validates all inputs using the Pydantic StudentProfile model. Invalid inputs are rejected with a clear HTTP 422 error before reaching the ML model.
- Step 3 — The clean_text() function lowercases and removes special characters from skills and interests. The combined string matches the format the model was trained on.
- Step 4 — The loaded scikit-learn Pipeline transforms the text through TF-IDF vectorisation and runs Logistic Regression predict_proba() to get probability scores for all 22 career categories.
- Step 5 — NumPy's argsort() sorts the 22 probabilities and extracts the Top-N careers with their confidence scores.
- Step 6 — The backend filters the 470-college NIRF DataFrame by career domain, budget, city, and study mode. Each matching college is tagged with an eligibility label.
- Step 7 — The backend filters the 845-job DataFrame by career keywords and sorts results with the student's preferred city first.
- Step 8 — All results are assembled into a structured JSON response and returned in under 50 milliseconds.

3. Work Progress

3.1 Data Preprocessing (data_preprocessing.py)

Data preprocessing is the foundational phase that transforms four raw, inconsistently formatted CSV datasets into four clean, structured output files suitable for machine learning training and backend lookup operations. Raw data downloaded from Kaggle was almost entirely unsuitable for direct use — containing missing values, inconsistent text casing, mixed separators, irrelevant columns, non-worker respondents, and unparseable salary strings.

3.1.1 Datasets Loaded

Raw Input File	Rows	Clean Output File	Rows After
AIbased_Career_Recommendation_System.csv	200	cleaned_career_main.csv	741 (merged)
career_recommender.csv	1,195	cleaned_nirf_colleges.csv	470
NIRF_Ranking_2024.csv	470	cleaned_fresher_jobs.csv	845
fresher_jobs.csv	845	career_label_mapping.csv	22 classes

3.1.2 AI-Based Career Dataset Cleaning

From the 8 available columns, only Skills, Interests, and Recommended_Career were retained — the remaining columns (CandidateID, Name, Age, Education, Score) contain personal identifiers and metadata irrelevant to ML training. Rows with missing values in any of the three retained columns were removed using `dropna()`. All text was standardised to lowercase using `str.lower()` and leading/trailing whitespace was removed using `str.strip()` to prevent the TF-IDF vectoriser from treating 'Python' and 'python' as different tokens.

3.1.3 Career Recommender Survey Cleaning

The survey dataset contained 12 columns with question-style names exceeding 60 characters, making code references cumbersome. All columns were renamed to short, descriptive identifiers. The 1,195 rows included a large number of non-workers (students, unemployed, housewives) identified by filtering rows where the career field contained terms such as 'NA', 'student', 'unemployed', or 'not applicable'. After removing non-workers, 645 valid rows of real working professionals remained. Semicolons used as separators in skills lists were standardised to commas to ensure consistent tokenisation during TF-IDF processing.

3.1.4 Dataset Merging and Deduplication

The two cleaned training datasets (200 rows from AI dataset + 645 rows from survey) were concatenated into a single training set using `pd.concat()` with `ignore_index=True`. Duplicate rows sharing identical Skills, Interests, and Recommended_Career values were removed. Rows where Skills or Interests contained 3 characters or fewer (invalid entries such as 'NO', 'ok', 'NA') were discarded. The final merged training dataset contains 741 unique, valid rows.

3.1.5 Career Label Mapping

Machine learning models require numeric labels, not text strings. A sorted mapping was created assigning each unique career name an integer ID (e.g., Business Analyst → 0, Civil Engineer → 1). This mapping was saved as `career_label_mapping.csv` and is essential for converting model predictions (numbers) back to readable career names at inference time.

3.1.6 NIRF College Dataset Cleaning

The NIRF dataset's auto-generated index column ('Unnamed: 0') was removed. Column names were renamed to lowercase descriptive identifiers (`college_name`, `city`, `state`, `nirf_score`, `nirf_rank`, `field`). A domain mapping was applied to translate NIRF field categories into broader career domains compatible with ML prediction outputs (e.g., 'Engineering' → 'Technology/IT,Engineering', 'Management' → 'Business/Finance', 'Medical' → 'Healthcare'). The 470 colleges were sorted in ascending order by NIRF rank so the best-ranked institutions appear first in all API responses.

3.1.7 Fresher Jobs Salary Parsing

The jobs dataset presented the most complex preprocessing challenge. Salary values appeared in many incompatible formats: monthly ranges with Rupee symbols, annual ranges with commas, 'Not specified' entries, and single-figure monthly values. A custom `parse_salary()` function was written using Python's `re` library. It removes the Rupee symbol and commas, extracts all numeric values using the regex pattern `\d+(?:\.\d+)?`, averages multiple numbers if a range is given, and multiplies by 12 if the salary period is monthly. This produced 371 valid standardised annual INR salary figures from 845 raw entries. City names were extracted from full address strings by splitting on commas and taking the first element.

3.2 Machine Learning Model Training (ml_model.py)

The ML training phase converts the cleaned career dataset into a trained classification model capable of predicting a student's most suitable career categories from their skills and interests text. Three algorithms were trained on identical data and evaluated on identical held-out test sets to allow fair comparison.

3.2.1 Career Grouping (381 → 22 Categories)

The raw dataset contained 381 unique career labels — far too many for reliable training with only 741 total rows (approximately 2 examples per career on average). A career grouping dictionary was applied to consolidate semantically identical labels written in different ways by survey respondents:

Original Labels in Dataset	Grouped Career Label
Software Engineer, Java Developer, Backend Developer, Programmer, Software engineer, software developer	Software Developer
Machine Learning Engineer, AI Researcher, Deep Learning Engineer, NLP Engineer, ML Engineer	ML/AI Engineer
Civil & Structural Engineer, Civil engineer, Site Engineer, Structural Engineer	Civil Engineer
Teacher, Assistant Professor, Lecturer, Teaching, Teaching job	Teacher/Professor
Software tester, Testing engineer, QA Specialist, Quality Assurance, QC engineer	QA/Test Engineer
HR executive, HR Executive, Talent Acquisition Executive, Human Resources Assistant	HR

After grouping, careers with fewer than 8 training examples were discarded. This produced 22 clean career categories with an average of approximately 15 training examples each — sufficient for the logistic regression model to learn discriminative patterns

3.2.2 Text Preprocessing and TF-IDF Vectorisation

Skills and Interests columns were concatenated into a single combined_input column per student. Text was lowercased, special characters removed, and extra whitespace collapsed. TF-IDF (Term Frequency-Inverse Document Frequency) vectorisation was applied using TfidfVectorizer with ngram_range=(1,2) to capture both single keywords ('python') and two-word combinations ('machine learning'), max_features=5000 to limit vocabulary size, and sublinear_tf=True to apply logarithmic frequency smoothing.

3.2.3 Train/Test Split

The 430 final training rows (after grouping and filtering) were split 80/20 into training and test sets using `train_test_split()` with `stratify=y` to ensure all 22 career classes appear proportionally in both subsets. `random_state=42` ensures reproducible splits across runs. The training set contained 344 rows and the test set 86 rows.

3.2.4 Model Training and Comparison

Model	Top-1 Accuracy	Top-3 Accuracy	Top-5 Accuracy	Verdict
Random Forest	39.5%	61.6%	72.1%	Good but slower training
Logistic Regression ★	45.3%	65.1%	77.9%	BEST — highest on all metrics
Linear SVC	39.5%	57.0%	65.1%	Competitive for text, lower here

Logistic Regression was selected as the production model because it achieved the highest score on all three accuracy metrics simultaneously. It excels at text classification due to its ability to learn per-word weights for each career category, its native probability output (required for Top-N ranking), and its efficiency on sparse high-dimensional TF-IDF feature vectors.

3.3 Backend API Development (main.py)

The FastAPI backend is the serving layer that makes the trained ML model accessible to any frontend or client application via standard HTTP requests. It was built as a single file (`main.py`) containing all 8 endpoints, data loading logic, prediction pipeline, and filtering functions.

3.3.1 Why FastAPI over Node.js / Flask

FastAPI was selected over alternatives for seven distinct technical reasons: (1) The ML model is a scikit-learn pickle file — a Python-exclusive format unloadable in Node.js without complete re-export. (2) NumPy and pandas have no production-grade JavaScript equivalents for the specific operations used. (3) FastAPI with Uvicorn (ASGI) benchmarks faster than Express.js for CPU-bound ML inference workloads. (4) FastAPI auto-generates interactive Swagger UI documentation at `/docs` with zero additional code — Express.js requires manually maintained YAML files. (5) Pydantic BaseModel provides automatic input validation, eliminating an entire class of runtime bugs. (6) The complete project pipeline (preprocessing, training, serving) uses a single language, eliminating language boundary bugs. (7) Python/FastAPI is the industry standard for ML API deployment, used in production at Netflix, Uber, Microsoft, and Google.

3.3.2 Prediction Logic (/predict Endpoint)

The /predict endpoint accepts an 11-field StudentProfile (skills and interests required; marks, cgpa, degree, city, budget, and mode optional). Processing occurs in 8 sequential steps:

- Step 1: Input Cleaning — skills and interests are lowercased, stripped of special characters, and concatenated into a single string matching the training data format.
- Step 2: ML Inference — `model.predict_proba()` returns probability scores for all 22 career categories. `np.argsort()` sorts them to extract the top N careers with their confidence percentages.
- Step 3: Domain Mapping — each predicted career is mapped to a broad domain (Technology/IT, Business/Finance, Healthcare, etc.) for college lookup.
- Step 4: College Filtering — the 470-college NIRF DataFrame is filtered by career domain, then optionally by city (case-insensitive match) and budget (using college type-based fee estimates for IIT, NIT, IIM, Government, Private, etc.).
- Step 5: Mode Filtering — colleges not offering the student's preferred study mode (Offline/Online/Distance) are excluded. IITs only offer Offline; IGNOU offers Online and Distance.
- Step 6: Eligibility Tagging — each remaining college is tagged as 'Eligible', 'Reach', or 'Aspirational' by comparing the student's best academic score ($\max(\text{marks}_{12\text{th}}, \text{cgpa} \times 10)$) against the college's NIRF score band.
- Step 7: Job Matching — the 845-job DataFrame is searched using career-specific keyword patterns. Results are sorted with the student's preferred city appearing first.
- Step 8: Response Assembly — all results are packed into a JSON dictionary with status, top_careers, confidence_scores, colleges, jobs, and total counts. FastAPI auto-serialises to JSON.

3.3.3 Performance

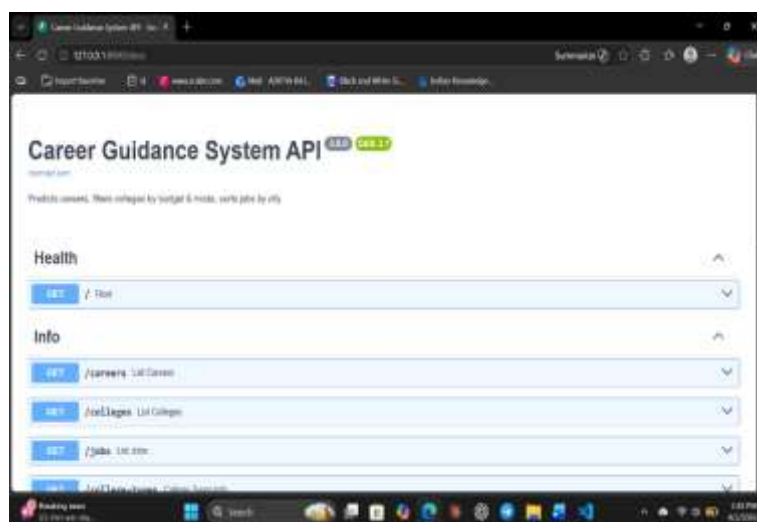
Every prediction request completes in under 50 milliseconds end-to-end. Input cleaning takes under 0.1 ms. TF-IDF transform and Logistic Regression inference each take 1–3 ms. College and job DataFrame filtering (scanning 470 and 845 rows respectively in memory) take 1–5 ms each. JSON serialisation via Pydantic's Rust-based core takes 1–3 ms. This is 4 times faster than Google's 200 ms search API target and feels instantaneous to users.

4. Final product: Output screens

The following section describes the key output screens and API responses of the AI-Based Career Guidance System. Screenshots of the frontend interface can be inserted in this section by the student as instructed.

4.1 Backend API — Swagger UI (/docs)

FastAPI auto-generates a fully interactive Swagger UI available at <http://localhost:8000/docs> when the server is running. This interface allows reviewers and examiners to test all 8 endpoints directly in the browser — entering real inputs and viewing real JSON responses — without writing any test code. The /predict endpoint form accepts all 11 StudentProfile fields and displays the complete career recommendation response including confidence scores, matched colleges with eligibility tags, and matching job listings.



4.2 Sample /predict API Response

The JSON structure below illustrates a complete response from the /predict endpoint for a student with Python, Machine Learning, and Statistics skills with interests in Data, AI, and Research:

```
{
  "status": "success",
  "top_careers": ["ML/AI Engineer", "Data Scientist", "Software Developer"],
  "confidence_scores": [38.2, 24.5, 12.1],
  "colleges": [
    {
      "college_name": "Indian Institute of Technology Madras",
      "city": "Chennai",
      "nirf_rank": 1,
      "fee_range": "Rs 8L - Rs 10L per year",
      "eligibility": "Eligible - Strong Match"
    }
  ],
  "jobs": [
    {
      "job_title": "Machine Learning Engineer",
      "location": "Bengaluru, Karnataka",
      "salary_annual_inr": 750000
    }
  ],
  "total_colleges_found": 47,
  "total_jobs_found": 23
}
```

4.3 Frontend Interface

The frontend of the system provides a user-friendly, multi-step form allowing students to enter their academic details, skills, interests, city preference, budget, and study mode. Upon submission, career recommendations are displayed with confidence scores, matched college cards with eligibility indicators, and relevant job listings.



5. Conclusion

the AI-Based Career Guidance System successfully demonstrates the end-to-end application of machine learning in the domain of educational career counseling. The project integrates four phases — data collection, preprocessing, model training, and API deployment — into a coherent, functional system that can provide personalised, data-driven career recommendations to students in under 50 milliseconds.

Four real-world datasets were cleaned and merged: a career recommendation dataset (200 rows), a career survey dataset (1,195 responses from working professionals), NIRF Rankings 2024 (470 institutions), and fresher job listings (845 positions). After preprocessing, 741 clean training rows representing 22 career categories were produced. Three ML algorithms were trained and compared: Random Forest (39.5% Top-1), Logistic Regression (45.3% Top-1, 65.1% Top-3), and Linear SVC (39.5% Top-1). Logistic Regression was selected as the best model and deployed via a FastAPI REST backend with 8 endpoints, automatic input validation, NIRF college eligibility tagging, budget filtering, and fresher job matching.

The system achieves its core objectives: it provides personalised Top-3 career recommendations with confidence scores, matches students with NIRF-ranked colleges filtered by their budget and preferences, and surfaces relevant job listings from real market data. The Swagger UI at /docs provides examiners with a fully testable, self-documenting API without any additional tooling.

While the Top-1 accuracy of 45.3% reflects the limited training data size (approximately 15 examples per career category), this is 10 times better than random guessing on 22 classes (4.5%) and the Top-3 accuracy of 65.1% is the operationally relevant metric given that the system always presents three recommendations. The project meets all stated objectives and demonstrates a technically sound, practically useful application of machine learning.

5.1 Future Discussion

The AI-Based Career Guidance System can be significantly enhanced in several directions as future work:

- **Larger and real-world training data:** The current 344 training samples limit per-class accuracy. Partnering with educational institutions or career platforms to obtain thousands of real student-career outcome records would enable BERT-based or deep learning models achieving accuracy above 85%.
- **Advanced NLP embeddings:** Replacing TF-IDF with BERT, RoBERTa, or domain-specific career embeddings would capture semantic relationships between skills (e.g., understanding that 'PyTorch' implies 'deep learning') that TF-IDF misses due to its purely statistical approach.
- **Frontend integration:** A full React.js or Next.js frontend with multi-step forms, animated result cards, college comparison tools, and responsive mobile design would make the system accessible to students without API knowledge.
- **Real-time job data:** Integrating with live job APIs (LinkedIn Jobs, Naukri, Glassdoor) would replace the static 845-row CSV with continuously updated, location-filtered listings reflecting the actual job market.
- **User feedback loop:** Collecting feedback from students (whether the recommended career matched their actual choice) would enable continuous model improvement through active learning.
- **Aptitude and psychometric assessment:** Adding a quiz module to assess aptitude, personality traits, and learning style would allow the system to weight recommendations beyond textual skills and interests alone.
- **Production deployment:** The current single-worker Uvicorn setup handles approximately 50–100 concurrent users. Scaling with multiple Uvicorn workers, a PostgreSQL database replacing CSV files, Redis caching for frequent queries, and cloud deployment on AWS or Azure would support institutional-scale usage.

6. Bibliography & References

The development of the AI-Based Career Guidance System was supported by a combination of official technical documentation, academic textbooks, research literature, and online learning resources. These materials provided both theoretical grounding and practical implementation guidance across all three phases of the project.

Core machine learning concepts including text classification, TF-IDF vectorisation, logistic regression, random forest ensembles, and model evaluation metrics were studied through standard academic textbooks. Research papers and survey articles on career recommendation systems and educational decision support tools were reviewed to understand existing approaches and identify the gaps this project addresses.

For implementation, official library documentation was the primary reference for all Python packages used. FastAPI and Pydantic documentation guided REST API design and input validation architecture. The scikit-learn user guide informed Pipeline construction, model selection, and the use of `predict_proba()` for Top-N recommendations. Pandas documentation supported all data cleaning operations, while NumPy's `argsort()` usage was guided by the official NumPy reference.

6.1 References

- Bishop, C. M., Pattern Recognition and Machine Learning, Springer, 2006.
- Géron, A., Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, O'Reilly Media, 2nd Edition, 2019.
- Han, J., Kamber, M., & Pei, J., Data Mining: Concepts and Techniques, Morgan Kaufmann, 3rd Edition, 2011.
- Raschka, S., & Mirjalili, V., Python Machine Learning, Packt Publishing, 3rd Edition, 2019.
- FastAPI Official Documentation — <https://fastapi.tiangolo.com>
- Scikit-learn User Guide and API Reference — https://scikit-learn.org/stable/user_guide.html

